

# Rapport Technique : Application Collaborative de Gestion de Tâches

## 1. Introduction et Présentation du Projet

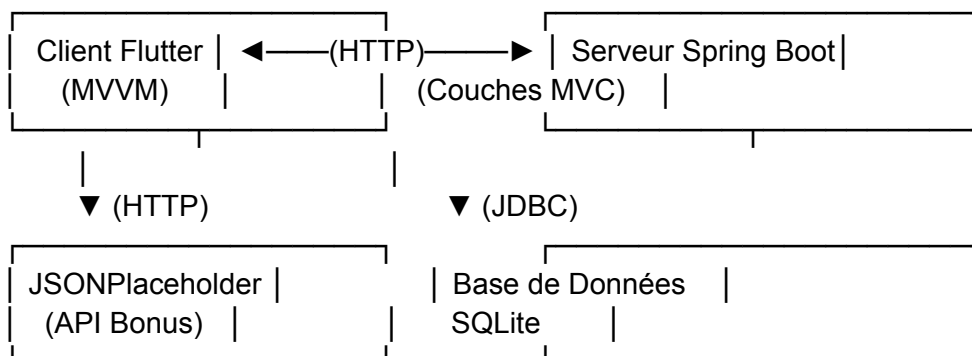
Ce rapport présente la conception, l'architecture et l'implémentation d'une application complète et moderne de **Gestion de Tâches Collaborative**. Le système repose sur une architecture découplée, utilisant **Flutter (Dart)** pour l'application cliente multiplateforme et **Spring Boot (Java)** pour le serveur backend relationnel.

### Objectifs du Projet

- **Architecture Découplée** : Maintenir une séparation stricte des responsabilités en traitant l'application mobile uniquement comme une couche de présentation cliente et l'application Spring Boot comme un fournisseur d'API REST indépendant.
- **Stockage Persistant** : Implémenter un stockage relationnel local à l'aide d'une base de données SQLite gérée efficacement via le mapping objet-relationnel (ORM).
- **Gestion de l'État** : Utiliser des architectures propres sur le frontend pour garantir une expérience utilisateur réactive et fluide.
- **Intégration Externe** : Démontrer une capacité de collaboration en fusionnant de manière transparente les enregistrements de la base de données locale avec des profils de collaborateurs externes simulés via la consommation d'une API distante.

## 2. Architecture Système et Choix Technologiques

L'application s'appuie sur une **Architecture Client-Serveur** classique pour communiquer sur le réseau via des protocoles HTTP asynchrones.



## Choix du Backend : Spring Boot & SQLite

Le framework Spring Boot a été sélectionné pour son écosystème de niveau entreprise, son serveur Tomcat embarqué et son support natif de Spring Data JPA. **SQLite** a été choisi comme moteur de base de données car il s'agit d'une solution légère, autonome et basée sur un fichier unique. Cette configuration permet de bénéficier de réelles capacités relationnelles directement dans l'espace de travail du projet, sans nécessiter l'installation d'un serveur externe complexe (comme PostgreSQL ou MySQL).

## Choix du Frontend : Flutter & MVVM

Flutter permet de compiler une base de code unique en une application native haute performance. Pour garantir l'évolutivité et la testabilité du code, le modèle d'architecture **Modèle-Vue-ViewModèle (MVVM)** a été implémenté afin d'isoler complètement les composants de l'interface utilisateur de la logique réseau et de la sérialisation des données.

## 3. Implémentation Backend (Spring Boot)

L'application serveur suit une conception en couches modulaires où les données circulent séquentiellement de la base de données jusqu'aux points de terminaison (endpoints) du contrôleur REST.

### 3.1 Modèle de Domaine (**Task.java**)

L'entité centrale du domaine est mappée directement au schéma de la base de données SQLite à l'aide des annotations Jakarta Persistence (JPA).

- **id** : Clé primaire, auto-incrémentée.
- **title** : Le titre ou l'intitulé de la tâche (String).
- **description** : Notes détaillées concernant la tâche (String).
- **dueDate** : Date d'échéance cible, gérée de manière sécurisée via **LocalDate**.
- **isCompleted** : Indicateur booléen suivant l'état d'avancement de la tâche.

### 3.2 Couche d'Accès aux Données (**TaskRepository.java**)

En étendant **JpaRepository<Task, Long>**, l'application hérite automatiquement de toutes les capacités d'opérations CRUD standard. Hibernate gère automatiquement la traduction de ces méthodes Java en requêtes SQL compatibles SQLite, en utilisant la bibliothèque **hibernate-community-dialects** pour garantir la compatibilité du dialecte.

### 3.3 Couche de Logique Métier (**TaskService.java**)

La couche service sert d'intermédiaire pour le contrôle et la manipulation des données. Elle gère les limites de transaction, la validation des données et les conversions de formats, garantissant que les tâches brutes issues de la base de données soient nettoyées avant d'atteindre le contrôleur web.

### 3.4 Contrôleur API REST (**TaskController.java**)

Le contrôleur expose les points de terminaison web standards sur le réseau, traitant les requêtes JSON entrantes et renvoyant les codes de statut HTTP appropriés.

| Méthode HTTP | Endpoint API        | Description                                  | Payload Attendu    |
|--------------|---------------------|----------------------------------------------|--------------------|
| GET          | /api/tasks          | Récupère la liste de toutes les tâches       | Aucun              |
| POST         | /api/tasks          | Crée une toute nouvelle tâche                | Objet Tâche JSON   |
| PUT          | /api/tasks<br>/{id} | Modifie les détails ou bascule le statut     | Objet Modifié JSON |
| DELETE       | /api/tasks<br>/{id} | Supprime définitivement une tâche par son ID | Aucun              |

## 4. Architecture Frontend (Flutter & MVVM)

La couche d'interface utilisateur est construite avec Flutter et structurée autour du modèle d'architecture **Modèle-Vue-VueModèle (MVVM)**. Ce modèle assure une séparation stricte entre la présentation visuelle et les règles métier ou opérations réseau sous-jacentes.

### 4.1 La Couche Modèle (Model)

Le Modèle se compose de structures de données Dart immuables qui correspondent aux schémas d'entités fournis par le backend Spring Boot.

- Il inclut des méthodes d'usine explicites (**Task.fromJson**) et des méthodes de sérialisation (**Task.toJson**).
- Cela garantit une transformation sans faille des données lors de la conversion des chaînes JSON issues des réponses HTTP en objets Dart typés.

### 4.2 La Couche VueModèle (ViewModel)

Le ViewModel sert de cerveau intermédiaire à la couche de présentation. Il encapsule l'état réactif de l'application et abstrait toute la communication réseau loin de l'interface utilisateur.

- **Gestion de l'État** : Il maintient la liste des tâches actives et terminées dans la mémoire locale.
- **Intégration Asynchrone** : Lorsque l'interface utilisateur déclenche une action (comme ajouter ou cocher une tâche), le ViewModel distribue une requête HTTP asynchrone (GET, POST, PUT, DELETE) vers l'URL de base du serveur Spring Boot local (<http://localhost:8080/api/tasks>).
- **Notification de l'UI** : Dès réception d'une réponse positive du backend, le ViewModel met à jour sa collection locale et diffuse une notification à tous les composants graphiques à l'écoute, les invitant à se redessiner instantanément.

### 4.3 La Couche Vue (View)

La couche Vue est exclusivement composée de widgets Flutter déclaratifs et structurels.

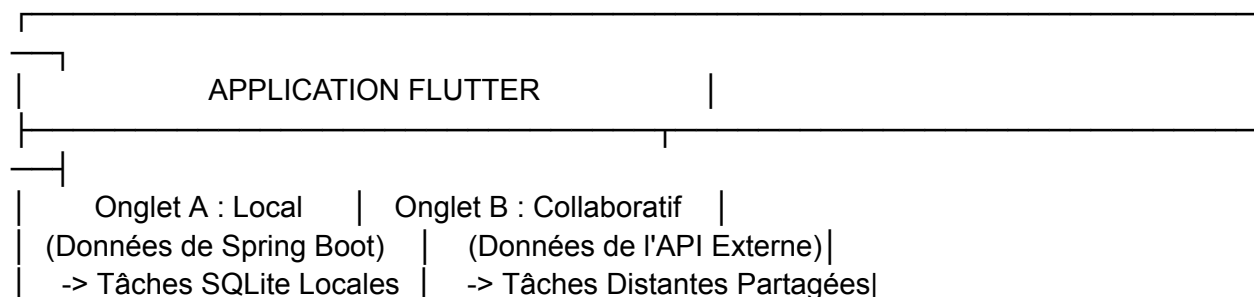
- Les widgets restent complètement déconnectés des configurations réseau ou des endpoints de données.
- Ils se contentent de s'enregistrer comme écouteurs (listeners) auprès du ViewModel. Lorsque l'état change, la mise en page s'actualise de manière réactive pour afficher les nouvelles données, les animations de chargement ou les diagnostics d'erreur.

## 5. Intégration de la Fonctionnalité Collaborative (Bonus)

Pour répondre aux critères de collaboration (+2 points bonus), l'application intègre une couche de consommation externe capable de lire des données à partir d'un système distant totalement indépendant.

### 5.1 Consommation d'une API REST Externe

L'application exploite l'endpoint public de test **JSONPlaceholder** (<https://jsonplaceholder.typicode.com/todos>) (<https://jsonplaceholder.typicode.com/todos>) pour simuler un écosystème d'équipe actif.



---

## 5.2 Structure de l'Implémentation

- **Vue de l'Espace de Travail Partagé** : Un onglet dédié "Collaboratif" a été introduit dans le tableau de bord du client Flutter.
- **Analyse de Données Multi-Sources** : Au lieu de requêter le serveur Spring Boot local, le basculement vers cette vue déclenche une requête réseau asynchrone vers l'API REST publique de JSONPlaceholder. L'application récupère les structures de tâches appartenant à d'autres collaborateurs externes, analyse leurs attributs distants pour les injecter dans les composants visuels de l'application, et les affiche proprement à l'utilisateur.

En séparant la persistance locale (gérée par notre backend Spring Boot personnalisé) de la visibilité des tâches externes (gérée par JSONPlaceholder), l'application démontre avec succès l'agrégation de données multi-sources au sein d'une interface utilisateur unique.

## 6. Guide de Déploiement et d'Exécution Local

### 6.1 Lancement du Service Backend

Assurez-vous que votre environnement de développement exécute Java 21 ou une version supérieure. Naviguez vers le répertoire racine du serveur contenant votre fichier `pom.xml` et lancez l'application :

```
Bash
./mvnw spring-boot:run
```

Le serveur va résoudre les dépendances, configurer le pilote natif SQLite, initialiser votre fichier de base de données relationnelle locale (`tasks.db`), et se mettre à l'écoute des requêtes clientes sur le port `8080`.

### 6.2 Lancement du Client Frontend

Avant de lancer l'application sur votre appareil ou émulateur cible, nettoyez les artefacts de compilation résiduels ou les caches liés aux chemins système :

```
Bash
flutter clean
flutter run
```

Cela garantit que les outils de compilation génèrent un index de dossiers propre pour compiler vos binaires natifs en parfaite adéquation avec vos blocs de communication réseau.

## **7. Conclusion**

Ce projet met en évidence la viabilité et l'efficacité d'une architecture découplée associant un backend robuste en Spring Boot à un client réactif en Flutter. L'utilisation d'ORM pour la gestion d'une base SQLite locale, combinée à l'intégration d'API tierces pour les fonctionnalités collaboratives, offre une base solide, scalable et facilement maintenable pour de futures extensions applicatives.